1 ARM Cortex-M3 Core

The documentation of a Cortex-M3 system is split into **three layers**, each defined by a different party. When looking for information, the first step is to know which layer answers the question.

- **Architecture (ARMv7-M)**: a **specification** written by ARM, not a physical chip. It fixes what software sees: the registers and their roles (the *programmer's model*), the instruction set, the way interrupts and faults are handled (the *exception model*), and the meaning of each region of the address space.
- **Core (Cortex-M3)**: the processor that ARM designs following that specification. It adds implementation details, such as the 3-stage pipeline, and a fixed set of *core peripherals*: the NVIC, the SysTick timer, the System Control Block, and an optional MPU.
- **Chip or board**: the physical product that a silicon vendor builds around the licensed core. The vendor decides how much Flash and SRAM exist, which device peripherals (UARTs, timers) are connected, and at which addresses.

Layer	Defined by	Examples of what it fixes
Architecture (ARMv7-M)	ARM	Registers, Thumb-2 instructions, exception numbers, address regions
Core (Cortex-M3)	ARM	NVIC, SysTick, SCB, MPU, 3-stage pipeline
Chip / board	Vendor	Memory sizes, device peripherals, IRQ assignment, clock

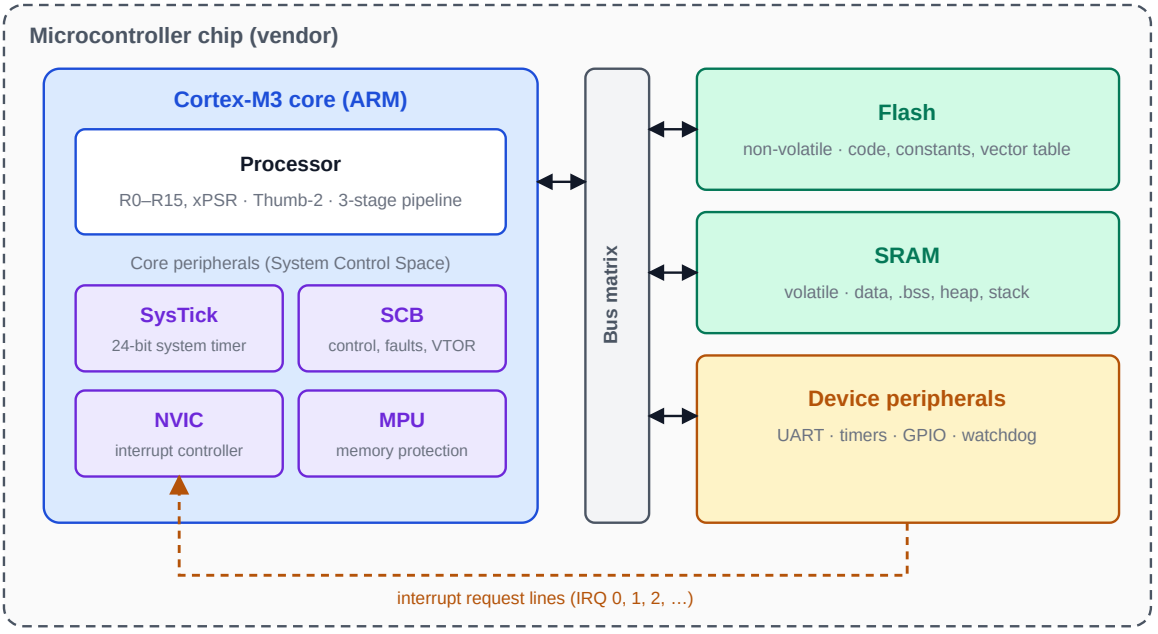


Figure 1.1: *Block diagram of a Cortex-M3 microcontroller: the ARM core with processor and core peripherals, the bus matrix, and the vendor's Flash, SRAM, and device peripherals.*

Everything in this chapter belongs to the first two layers and is therefore identical on every Cortex-M3. Board-specific values are taken from the `mps2-an385` platform (an FPGA image of a Cortex-M3 system, emulated by QEMU, clocked at 25 MHz) and are marked as such.

2 Programmer's model: registers and ISA

The programmer's model is the set of registers visible to software and to the debugger. At this level there are **no variables**, only registers and memory: a C statement such as `x = x + 1` becomes "load `x` into a register, add 1, store it back".

The registers of the Cortex-M3 fall into two groups:

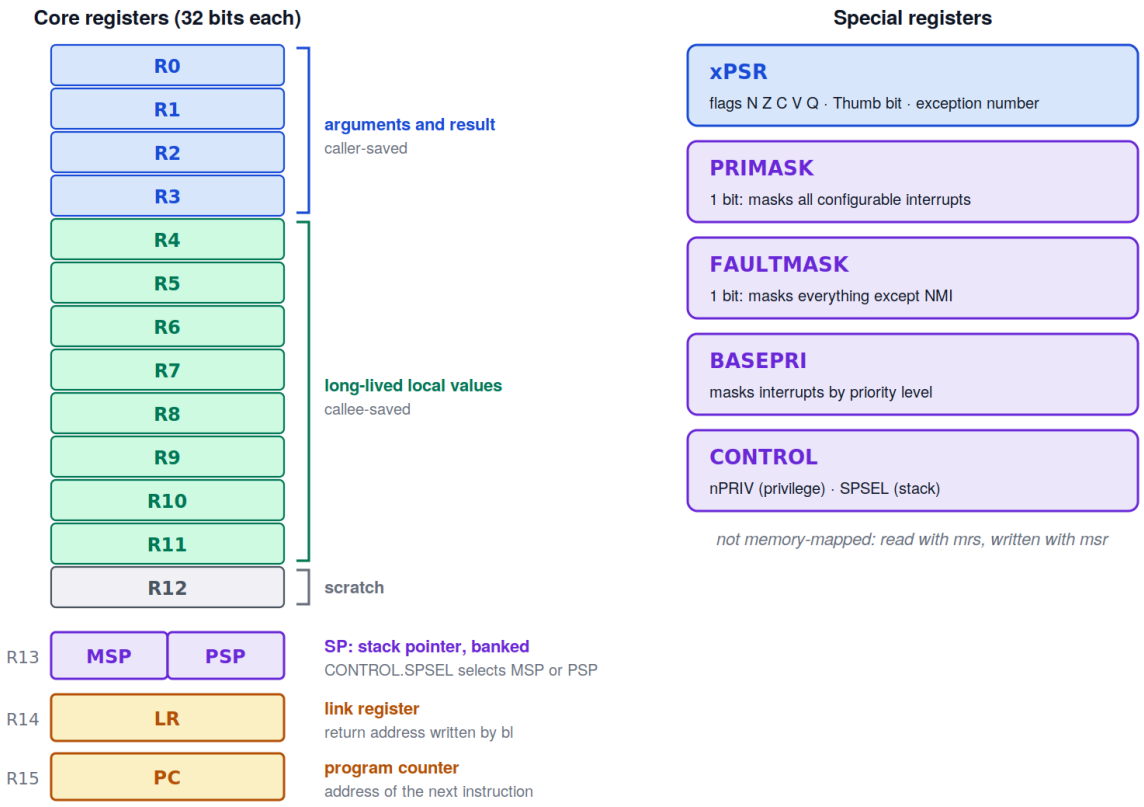
- **Core registers** (`R0` – `R15`): they are the operands of ordinary instructions, such as `add r0, r1, r2` or `ldr r3, [r0]` .
- **Special registers** (`xPSR` , `PRIMASK` , `FAULTMASK` , `BASEPRI` , `CONTROL`): they describe and control the state of the processor, and cannot be used as operands; they are read and written only with the dedicated instructions `mrs` and `msr` .

None of them has a memory address: they live inside the CPU. All of them are **32 bits** wide.

2.1 Core Registers

Register	Name	Role
`R0` – `R12`	General purpose	Operands and addresses; `R0` – `R3` carry function arguments
`R13`	`SP`	Stack pointer: address of the top of the stack
`R14`	`LR`	Link register: return address after a function call
`R15`	`PC`	Program counter: address of the next instruction
—	`xPSR`	Program status: flags, Thumb bit, current exception number

The stack is **full descending**: it grows towards lower addresses, and `SP` points to the last occupied word. A push first decrements `SP` by 4 and then writes at the new `SP` . `R13` is a banked name: two physical stack pointers exist (`MSP` and `PSP`), and `SP` always refers to the one currently selected (see the section on execution modes).



Cortex-M3 registers: *thirteen general-purpose registers with their AAPCS roles, the banked stack pointer, the link register, the program counter, and the special registers accessed with mrs and msr.*

2.2 Special Registers

Special registers hold the **state of the processor** rather than data. They cannot be used as operands of ordinary instructions such as `add` or `ldr`, and they have **no memory address**. They are moved to and from a core register with two dedicated instructions:

```
mrs r0, primask     @ move register from special: r0 = PRIMASK
msr control, r0     @ move special from register: CONTROL = r0
```

The names follow the direction of the copy: `mrs` copies a **special** register into a core **register**, `msr` copies a core **register** into a **special** register. To change a single bit, the usual read-modify-write sequence is needed: `mrs` to read, `orr` or `bic` to modify, `msr` to write back.

Special register	Content	Described in
`xPSR`	Program status: flags, Thumb bit, current exception number	The Program Status Register xPSR
`CONTROL`	Thread-mode privilege and stack selection	The CONTROL Register
`PRIMASK`	1 bit: masks every exception with configurable priority	The PRIMASK Register
`FAULTMASK`	1 bit: also masks HardFault; only NMI is taken	—

Special register	Content	Described in
`BASEPRI`	Masks exceptions whose priority value is greater than or equal to it	—

`FAULTMASK` and `BASEPRI` are listed for completeness: the firmware described in these notes does not use them.

> **IMPORTANT**
>
> Writing `CONTROL`, `PRIMASK`, `FAULTMASK`, and `BASEPRI` is reserved to privileged code. In unprivileged code the write is silently ignored: no fault is raised, the register simply keeps its value.

2.2.1 The Program Status Register xPSR

`xPSR` holds neither data nor addresses: it **describes** the current **state of the processor**. The name reflects that it is the union of three sub-registers occupying disjoint bits of the same 32-bit word.

Sub-register	Bits	Content
`APSR` (Application)	31–28	Flags `N` (negative), `Z` (zero), `C` (carry), `V` (overflow)
`EPSR` (Execution)	24	`T`: Thumb state, always 1 on Cortex-M
`IPSR` (Interrupt)	8–0	Number of the exception being handled, 0 if none



APSR: condition flags · EPSR: Thumb state (always 1) · IPSR: running exception (0 = Thread mode)

Figure 1.2: *Bit layout of xPSR: N, Z, C, V flags in APSR, Thumb bit in EPSR, exception number in IPSR.*

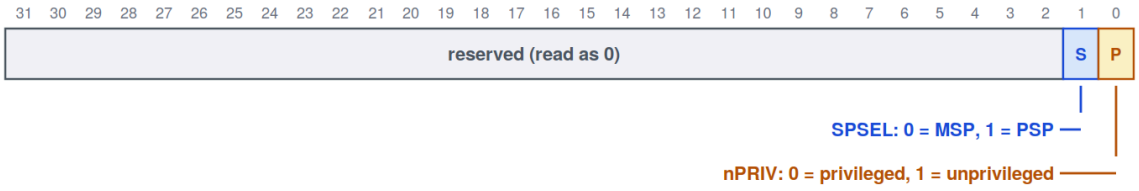
The flags are written by comparison and arithmetic instructions and read by conditional branches; this is how every `if` and loop condition is implemented at machine level.

> **EXAMPLE**
>
> Flags in action `cmp r1, r2` computes `r1 - r2`, discards the result, and updates only the flags: `Z = 1` if the operands are equal, `C = 1` if `r1 >= r2` as unsigned values. The following `bhs done` reads `C` and branches if it is 1.

`IPSR` identifies the execution mode: 0 means Thread mode (ordinary code), any other value is the number of the exception whose handler is running.

2.2.2 The CONTROL Register

`CONTROL` decides **how ordinary code runs**: with or without privileges, and on which of the two stacks. Only its **two lowest bits are used** on the Cortex-M3; the other bits are reserved and read as 0.



both bits act only in Thread mode; Handler mode is always privileged and always uses MSP

CONTROL register: *bit 1 (SPSEL) selects the Thread-mode stack, bit 0 (nPRIV) the Thread-mode privilege; bits 31–2 are reserved.*

Bit	Name	Meaning
0	`nPRIV`	0 = privileged, 1 = unprivileged (Thread mode only)
1	`SPSEL`	Thread-mode stack pointer: 0 = `MSP`, 1 = `PSP`

The two bits give four possible values:

`CONTROL`	Thread privilege	Thread stack	Typical use
`0`	Privileged	`MSP`	State after reset: boot code and kernel
`1`	Unprivileged	`MSP`	Rarely used
`2`	Privileged	`PSP`	Rarely used
`3`	Unprivileged	`PSP`	Application task with its own stack

Both bits act only in **Thread mode**, the mode of ordinary code. Exception handlers run in Handler mode, which is always privileged and always uses `MSP`, whatever `CONTROL` contains. The meaning of the two modes, of privilege, and of the two stacks is described in the section on execution modes, privilege, and stacks.

`CONTROL` is changed with a read-modify-write on a special register, followed by an `isb` instruction:

```
mrs r0, control     @ r0 = CONTROL
orr r0, r0, #1      @ set nPRIV
msr control, r0     @ CONTROL = r0: Thread mode is now unprivileged
isb                 @ apply the new setting to the following instructions
```

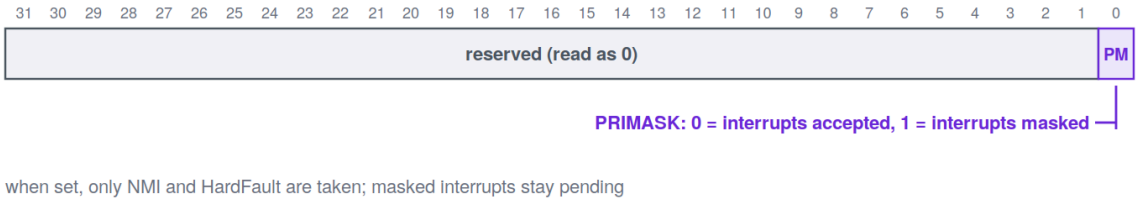
The `isb` (instruction synchronization barrier) is needed because the CPU has already fetched the next instructions under the old setting; `isb` discards them, so that every following instruction runs with the new value.

> **IMPORTANT**
>
> A one-way change Only privileged code can write `CONTROL`. Once Thread mode has become unprivileged, a `msr control` that tries to clear `nPRIV` is silently ignored: the only ways back to privileged execution are a reset or an exception, whose handler runs privileged in Handler mode.

2.2.3 The PRIMASK Register

`PRIMASK` is a one-bit register that **switches interrupts off and on**. Only bit 0 is used; the other bits are reserved

and read as 0.



when set, only NMI and HardFault are taken; masked interrupts stay pending

PRIMASK register: *bit 0 masks every exception with configurable priority; bits 31–1 are reserved.*

PRIMASK	Effect
`0`	Interrupts are accepted normally (value after reset)
`1`	Every exception with configurable priority is held back: device interrupts, SysTick, PendSV, SVCall. Only NMI and HardFault can still be taken

Masked interrupts are **not lost**: they remain *pending*, and the CPU takes them as soon as `PRIMASK` returns to 0. Exceptions, priorities, and pending state are described in the section on the exception model.

> **WARNING**
>
> `svc` with interrupts masked SVCall is the exception that cannot wait: `svc` is a synchronous request and cannot stay pending. Executed while `PRIMASK` is 1, it escalates to HardFault.

`PRIMASK` is usually changed with two dedicated instructions, which do not need a core register:

Assembly	C (CMSIS)	Effect
`cpsid i`	`__disable_irq();`	`PRIMASK = 1`: interrupts masked
`cpsie i`	`__enable_irq();`	`PRIMASK = 0`: interrupts accepted

The main use of `PRIMASK` is a *critical section*: a short piece of code that must run without being interrupted, for example while updating a variable that an interrupt handler also uses. Boot code also masks interrupts at the start and unmasks them only once the vector table, handlers, and priorities are configured.

A critical section must restore the **previous** value of `PRIMASK`, not simply unmask interrupts. If a critical section runs inside another one, an unconditional `cpsie i` at its end would re-enable interrupts while the outer section is still running. The safe pattern saves `PRIMASK` with `mrs` and restores it with `msr`:

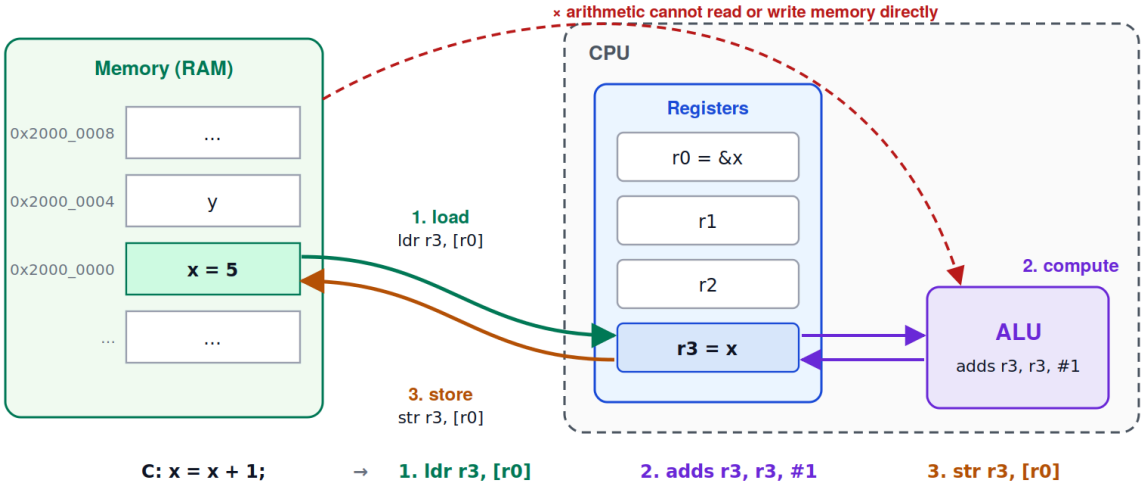
```
mrs   r0, primask   @ save the previous value (0 or 1)
cpsid i             @ mask interrupts
...                 @ critical section
msr   primask, r0   @ restore the previous value
```

> **IMPORTANT**
>
> Only privileged code can change `PRIMASK`. In unprivileged code `cpsid i`, `cpsie i`, and `msr primask` are silently ignored, so an application task cannot switch interrupts off.

2.3 Load/Store Architecture

The Cortex-M3 has a **load/store instruction set**: arithmetic and logic operate **only on registers**, and memory is accessed exclusively through load (`ldr` , memory to register) and store (`str` , register to memory) instructions. **Every operation** on a variable therefore **passes through a register**.

Two rules help to read every instruction below. The first operand is usually the destination (`ldr r3, …` writes `r3`), except in stores, where the first register is the value to save. Square brackets `[ ]` always mean "**the memory at this address**".



Load/store architecture: *memory is reached only through loads and stores; arithmetic works on registers. The C statement x = x + 1 needs three instructions.*

2.3.1 Load/Store Instructions

Load and store instructions are the only instructions that **access memory**. A *load* copies a value from memory into a register, a *store* copies a register into memory. The address is always taken from a register, written in square brackets, and can be adjusted by an offset (`[r0, #7]`) or updated after the access (`[r0], #4`). Suffixes select the size of the access: no suffix for a 32-bit word, `b` for a byte, `h` for a halfword (16 bits).

Assembly	C equivalent	Explanation
`ldr r3, [r0]`	`r3 = *r0;`	*Load register*: read the 32-bit word at address `r0` into `r3`
`str r3, [r1]`	`*r1 = r3;`	*Store register*: write `r3` to memory at address `r1`
`ldr r3, [r0], #4`	`r3 = *r0; r0 += 4;`	Load, then add 4 to `r0` so it points to the next word: used to walk arrays
`strb r1, [r0, #7]`	`((uint8_t *)r0)[7] = r1;`	*Store byte*: write only the low byte of `r1` at `r0 + 7` ; `r0` does not change
`ldr r0, =0x20400000`	`r0 = 0x20400000;`	Pseudo-instruction: load a 32-bit constant written as a number, through the literal pool

Assembly	C equivalent	Explanation
`ldr r0, =_estack`	`r0 = (uint32_t)&_estack;`	Same as above, with the value given by a symbol: the linker replaces `_estack` with `0x20400000`

2.3.1.1 LOADING A 32-BIT CONSTANT: THE LITERAL POOL

A Thumb instruction is 16 or 32 bits long. These bits must hold the operation and the registers, so there is no space left for a full 32-bit value such as the address `0x20400000`. The constant must therefore be stored in memory and loaded from there.

The pseudo-instruction `ldr r0, =0x20400000` does this automatically. The assembler:

1. writes the value `0x20400000` in memory, as data, just after the code; this small area of constants is called the *literal pool*;
2. replaces the pseudo-instruction with a normal load that reads that word. The load gives the position of the word as a distance from the current instruction, so a disassembler shows it as `ldr r0, [pc, #4]`, which means "read the word 4 bytes ahead of here".

```
0x48:  ldr r0, [pc, #4]     reads the word at 0x50 into r0
0x4A:  ...                  rest of the code
0x4E:  b   elsewhere        the CPU jumps away before reaching 0x50
0x50:  0x20400000           literal pool: a constant, not an instruction
```

The CPU must never execute the literal pool: if it did, it would read the bytes of `0x20400000` as an instruction. The directive `.ltorg` tells the assembler where to put the pool, and it is placed where execution never arrives, for example right after an unconditional branch.

2.3.2 Data-Processing Instructions

Data-processing instructions compute on registers: arithmetic (`add`, `sub`), logic (`and`, `orr`), bit shifts, copies between registers, and comparisons. They **never touch memory**. The destination is the first operand; the last operand can be a register or a small constant written with `#`. A trailing `s` (as in `movs`) also updates the flags of `xPSR`, and comparison instructions do nothing else.

Assembly	C equivalent	Explanation
`movs r3, #0`	`r3 = 0;`	*Move*: put a small constant in `r3` and update the flags
`mov sp, r0`	`sp = r0;`	*Move*: copy one register into another, here into the stack pointer
`orr r1, r1, #0x10`	`r1 |= 0x10;`	Bitwise *OR*: set bit 4 of `r1`, leave the other bits unchanged
`cmp r1, r2`	(compute `r1 - r2`)	*Compare*: subtract, update the flags with the result, then discard the result

2.3.3 Branch Instructions

Branch instructions **change the flow of execution** by **loading a new address into the program counter**. An unconditional branch always jumps; a conditional branch jumps only if the flags satisfy its condition, which is

written as a suffix (`eq` , `ne` , `hs` , `lt` …) and usually tests the result of a previous `cmp` . A *branch with link* also saves the return address in `lr` , which is how function calls work.

Assembly	C equivalent	Explanation
`b label`	`goto label;`	*Branch*: always jump to `label`
`bhs done`	`if (r1 >= r2) goto done;`	*Branch if higher or same*: jump only if the last `cmp` found `r1 >= r2` (unsigned)
`bl main`	`main();`	*Branch with link*: save the return address in `lr` , then jump; return is `bx lr`

2.3.3.1 WRITING A LOOP: COMPARE AND BRANCH

Assembly has no `while` or `for` . A loop is built from three pieces: a compare, a conditional branch that leaves the loop, and an unconditional branch that goes back to the start.

The C loop:

```
while ( r1 < r2 ) {
    /* body */
}
```

becomes:

```
loop:   cmp   r1, r2        @ 1. compare r1 with r2
        bhs   done          @ 2. if r1 >= r2, jump to done (exit)
        ...                 @ 3. body of the loop
        b     loop          @ 4. jump back to step 1
done:                       @ code after the loop
```

`loop:` and `done:` are *labels*: names given to code addresses, used as targets of branches.

- **Step 1**: `cmp` subtracts `r2` from `r1` , throws the result away, and only updates the flags, which record how the comparison went.
- **Step 2**: `bhs` (*branch if higher or same*) looks at the flags and jumps to `done` if `r1 >= r2` . It tests the opposite of the C condition, because it decides when to *leave* the loop.
- **Step 3**: if the branch is not taken, the CPU simply continues with the next instructions, which are the body of the loop.
- **Step 4**: `b` always jumps back to `loop` , so the comparison is repeated before every new iteration.

2.3.4 Special Registers Access Instructions

Special registers cannot be used as operands of ordinary instructions, so a separate group of instructions moves them to and from core registers. `cpsid` and `cpsie` are shortcuts that change only `PRIMASK` , without going through a core register. Writing a special register is allowed only in privileged code; in unprivileged code the write is silently ignored.

Assembly	C equivalent	Explanation
`mrs r0, control`	`r0 = CONTROL;`	*Move register from special*: copy a special register into a core register
`msr control, r0`	`CONTROL = r0;`	*Move special from register*: copy a core register into a special register
`mrs r0, ipsr`	`r0 = IPSR;`	Read one part of `xPSR`, here the number of the running exception
`cpsid i`	`__disable_irq();`	*Change processor state, interrupts disable*: set `PRIMASK` to 1, masking interrupts
`cpsie i`	`__enable_irq();`	*Change processor state, interrupts enable*: clear `PRIMASK` to 0

2.4 Thumb State and the Thumb Bit

The Cortex-M executes only the Thumb-2 instruction set, a mix of 16-bit and 32-bit instructions. This has a side effect on code addresses:

- **Bit 0 is free**: instructions always start at an even address, so bit 0 of a code address is always 0 and carries no information.
- **ARM reuses it as the Thumb bit**: every code pointer the CPU jumps to (a vector table entry, a function pointer, the target of `bx`) must have bit 0 set to 1.
- **The CPU clears it before jumping**: it copies the bit into `xPSR.T`, sets it to 0, and jumps to the even address.
- **Example**: a handler located at `0x40` is stored in the vector table as `0x41`.

> ⚠ **THUMB BIT EQUAL TO 0**
>
> Branching to an address with bit 0 clear requests ARM state, which the Cortex-M does not implement. The result is a UsageFault (`INVSTATE`), not an ordinary crash at the target address.

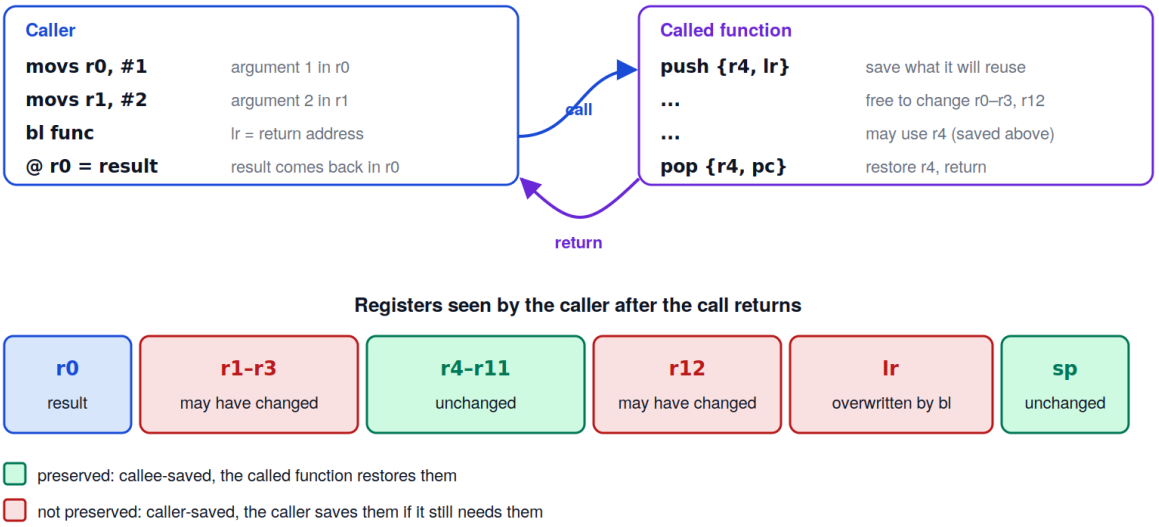
2.5 Calling Convention (AAPCS)

The **AAPCS (Procedure Call Standard for the Arm Architecture)** defines how functions **exchange arguments** and **which registers** they must preserve. Compiled C and hand-written assembly interoperate only if both follow it, and the exception model is designed around it.

Register	Role in a call	Preserved across the call
`r0` – `r3`	First four arguments; `r0` also returns the result	No (caller-saved)
`r4` – `r11`	Long-lived local values	Yes (callee-saved)
`r12`	Scratch	No (caller-saved)
`lr`	Return address written by `bl`	No; a non-leaf function saves it
`sp`	Multiple of 8 at every public call	Yes

Registers are shared: when a function calls another one, the called function may overwrite values the caller still needs. The AAPCS splits the responsibility for saving them:

- `r0` – `r3` **and** `r12` **are the caller's problem.** The called function may change them freely. If the caller still needs one of these values after the call, it must save it before calling.
- `r4` – `r11` **are the called function's problem.** The caller finds them unchanged after the call. A function that wants to use `r4` must save its old value at entry and restore it before returning, typically with `push {r4, lr}` at entry and `pop {r4, pc}` at exit; the same `pop` also returns, because it loads the saved `lr` into `pc`.
- **A leaf function may save nothing.** A *leaf function* calls no other function, so `lr` is never overwritten. If it also uses only `r0` – `r3` and `r12`, it has nothing to restore and does not touch the stack at all.



AAPCS function call: *arguments travel in r0–r3 and the result in r0; the called function restores r4–r11 and sp, while r0–r3, r12, and lr may be changed by the call.*

EXAMPLE

A leaf function: `add3` Consider a minimal C function that adds three numbers:

```
int add3( int a, int b, int c )
{
    return a + b + c;
}
```

Following the AAPCS, the arguments `a` , `b` , `c` arrive in `r0` , `r1` , `r2` , and the result must be returned in `r0` . The compiler translates the function into three instructions:

```
add3:
    adds r0, r0, r1     @ r0 = a + b
    adds r0, r0, r2     @ r0 = r0 + c
    bx   lr             @ return; the result is in r0
```

A call such as `r = add3( 1, 2, 3 );` places the arguments in registers and jumps to the function:

```
    movs r0, #1         @ a = 1
    movs r1, #2         @ b = 2
    movs r2, #3         @ c = 3
    bl   add3           @ lr = return address, jump to add3
                        @ back here: r0 = 6
```

`add3` is a *leaf function*: it calls no other function, so `lr` is never overwritten, and it uses only `r0` – `r2` , which are caller-saved. It therefore saves nothing: no `push` , no `pop` , no use of the stack, and it returns with a simple `bx lr` .

3 Address Space and Memory-Mapped Registers

The Cortex-M3 sees a single 4 GB physical address space (32-bit addresses). The architecture divides it into regions and fixes the purpose of each one; the **vendor decides how much memory or which devices actually populate them**.

There is **no address translation** (no MMU, hence no virtual memory): the address issued by an instruction is the address seen by memory and peripherals, and no run-time loader chooses where a program is placed. All placement decisions are made at link time and recorded in the image.

3.1 Address Space Overview

Address range	Region	Purpose	Populated on `mps2-an385`
`0x0000_0000` – `0x1FFF_FFFF`	Code	Program memory: code, constants, vector table at `0x0`	4 MB at `0x0` ("FLASH")
`0x2000_0000` – `0x3FFF_FFFF`	SRAM	Data memory: variables, heap, stack	4 MB at `0x2000_0000` ("RAM")
`0x4000_0000` – `0x5FFF_FFFF`	Peripherals	Registers of the on-chip devices added by the vendor	UARTs, timers, watchdog
`0x6000_0000` – `0xDFFF_FFFF`	External RAM / devices	Memory and devices outside the chip	Not used
`0xE000_0000` – `0xE00F_FFFF`	Private Peripheral Bus	Registers of the core peripherals (NVIC, SysTick, SCB, MPU) and debug units	System Control Space used by the firmware
`0xE010_0000` – `0xFFFF_FFFF`	Vendor-specific	System area reserved for the vendor	Not used in these notes

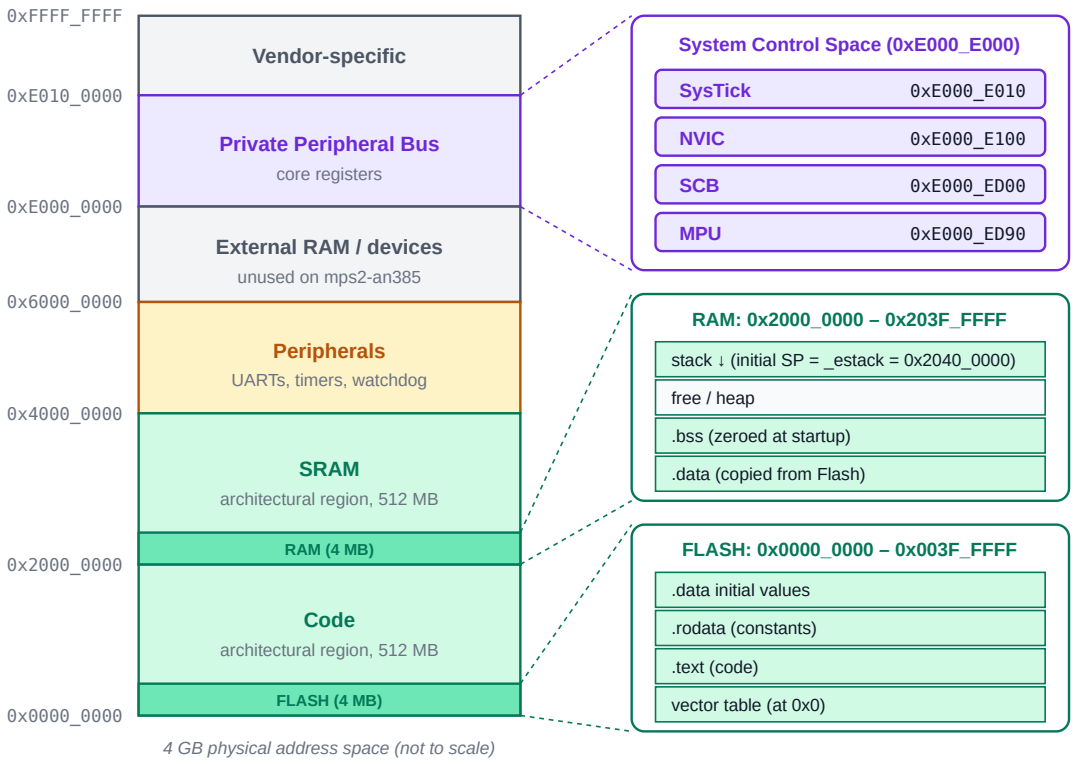


Figure 1.3: *Cortex-M3 address space with the regions populated on mps2-an385 and the System Control Space in detail.*

A region is only a range of addresses: memory or registers exist only where the chip connects something, and an access to an empty address raises a BusFault. On `mps2-an385` only a small part of the 4 GB is populated:



The External RAM / devices and Vendor-specific regions are not used on this board, so the following sections cover only the three populated regions, from the lowest address to the highest.

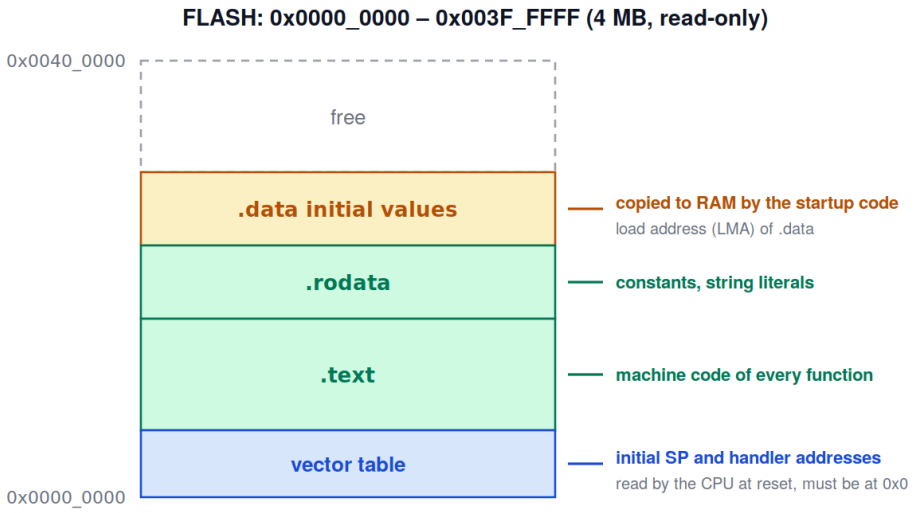
3.2 Code and SRAM: the Memories

The two lowest regions hold the **memories of the chip**. On `mps2-an385`, 4 MB of program memory ("**FLASH**") sit at the start of the Code region, and **4 MB** of data memory ("**RAM**") at the start of the **SRAM region**.

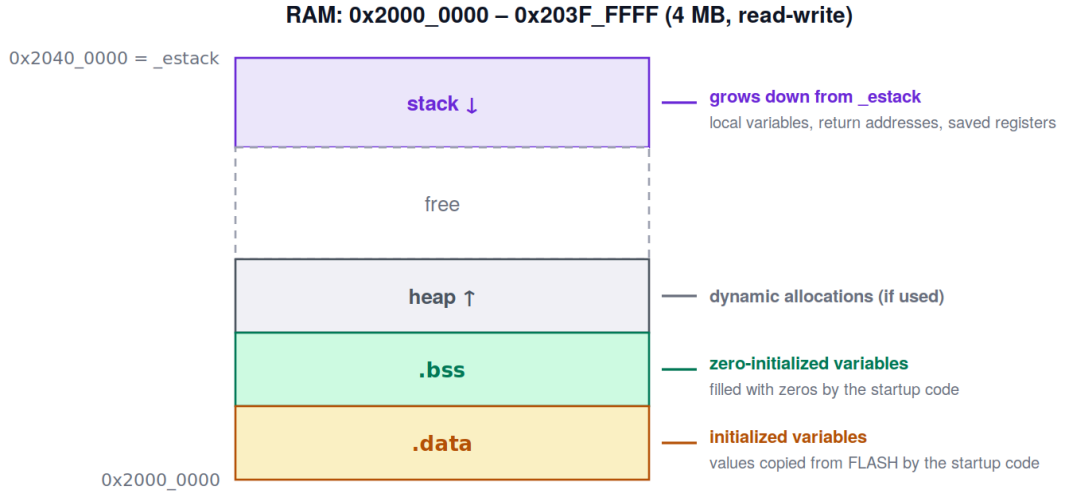
- **Flash** keeps its contents without power. It holds the vector table, the code, the constants, and the initial values of initialized variables.

- **SRAM** is fast writable memory, but its contents are undefined at power-on. It holds the variables (initialized and zeroed), the heap, and the stack.

As a consequence, before C code can run safely, the startup code must copy the initial values of variables from Flash to SRAM and fill the zeroed variables with zeros. Which part of the program goes into which memory is decided by the **linker script**, described in the chapter on boot and CPU configuration.



FLASH contents: *the vector table at address 0x0, the code, the constants, and the initial values of .data.*

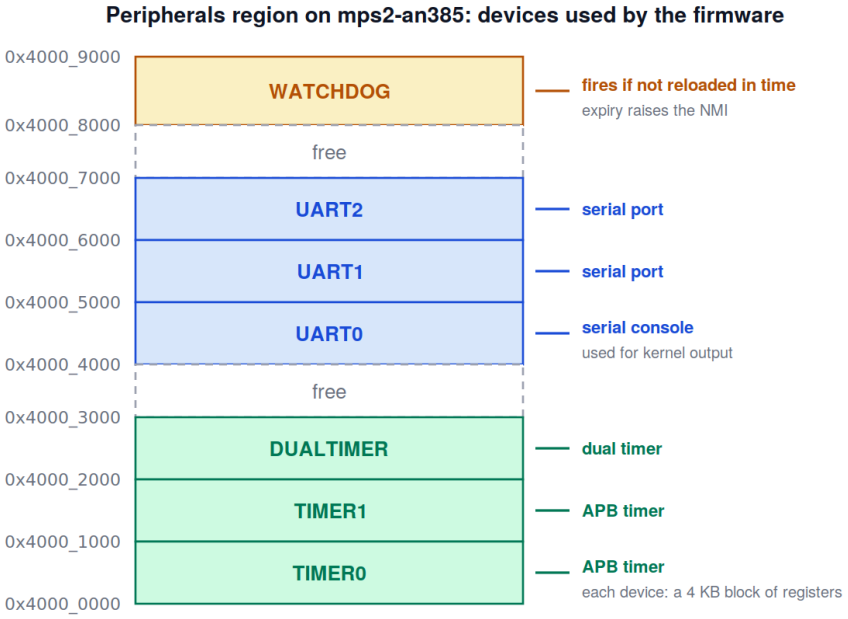


RAM contents: *the variables of .data and .bss at the bottom, the heap above them, and the stack growing down from the top.*

> **INFO**
>
> Emulated flash On `mps2-an385` , and in QEMU, the 4 MB at address `0x0` are fast RAM preloaded with the program image. They play the role of Flash and are treated as read-only, exactly like real Flash on a microcontroller.

3.3 Peripherals: Device Registers

The Peripherals region holds the registers of the devices that the vendor adds around the core: **timers, UARTs, GPIO,** and so on. Each device occupies a small block of addresses, often **4 KB**, and is controlled by reading and writing its registers. Which devices exist, and at which addresses, depends entirely on the chip: on `mps2-an385` they include three UARTs, two timers, a dual timer, and a watchdog, shown in the memory map above. These devices and their drivers are described in the chapter on devices, interrupts, and system calls.



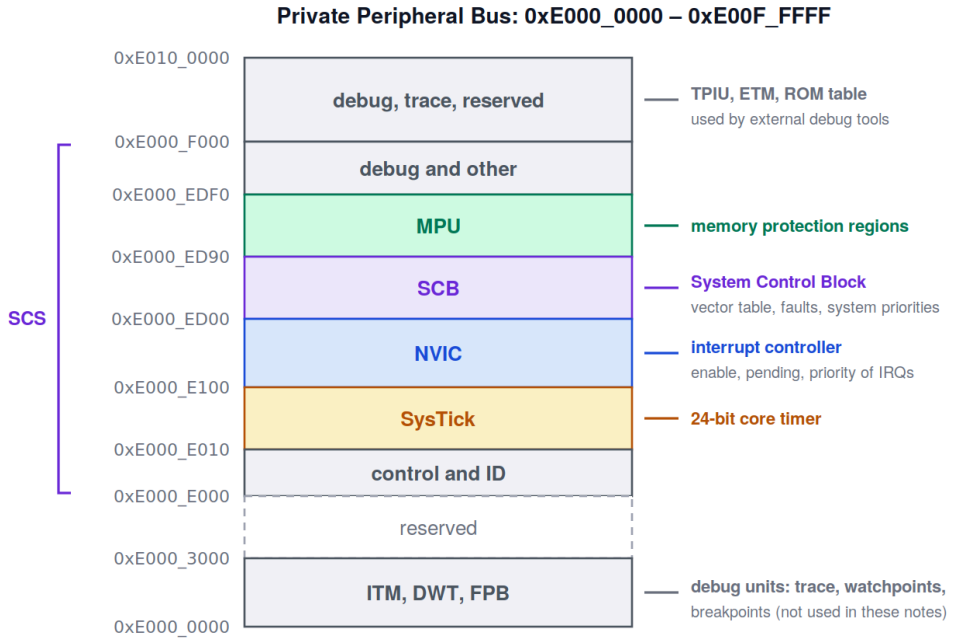
Peripherals contents: *on mps2-an385 each device occupies a 4 KB block of registers; only part of the 512 MB region is used.*

3.4 PPB: the Core Peripherals

The **Private Peripheral Bus** holds the registers of the core peripherals, together with debug and trace units that are not used in these notes. Inside it, the **System Control Space** (SCS, `0xE000_E000` – `0xE000_EFFF`) groups the four core peripherals, which are identical on every Cortex-M3 regardless of the vendor.

Core peripheral	Base address	Purpose
SysTick	`0xE000_E010`	24-bit periodic timer
NVIC	`0xE000_E100`	Enabling, pending state, and priority of device interrupts

Core peripheral	Base address	Purpose
SCB (System Control Block)	`0xE000_ED00`	Vector table location, fault configuration and status, system priorities
MPU	`0xE000_ED90`	Memory protection regions



PPB contents: *the System Control Space holds the registers of the four core peripherals; the rest of the bus belongs to debug and trace units.*

System Control Space (privileged access only)

0xE000_EFFF

Other 528 B

debug and other registers

0xE000_EDF0

0xE000_EDEF

MPU 96 B

memory protection unit

0xE000_EDA0 ◀ **RASR** region size and permissions

0xE000_ED9C ◀ **RBAR** region base address

0xE000_ED98 ◀ **RNR** region number

0xE000_ED94 ◀ **CTRL** enable, PRIVDEFENA

0xE000_ED90 ◀ **TYPE** number of regions

0xE000_ED90

0xE000_ED8F

SCB 144 B

System Control Block

0xE000_ED38 ◀ **BFAR** BusFault address

0xE000_ED34 ◀ **MMFAR** MemManage fault address

0xE000_ED2C ◀ **HFSR** cause of HardFault

0xE000_ED28 ◀ **CFSR** cause of configurable faults

0xE000_ED24 ◀ **SHCSR** enable fault handlers

0xE000_ED18 ◀ **SHPR1** system exception priorities (SHPR1–3)

0xE000_ED14 ◀ **CCR** fault traps, stack alignment

0xE000_ED0C ◀ **AIRCR** reset, priority grouping

0xE000_ED08 ◀ **VTOR** vector table address

0xE000_ED04 ◀ **ICSR** pending state, PendSV trigger

0xE000_ED00 ◀ **CPUID** core identification

0xE000_ED00

0xE000_ECFF

NVIC 3 KiB

interrupt controller

0xE000_E400 ◀ **IPR** priority, 1 byte per IRQ

0xE000_E300 ◀ **IABR** active (handler running)

0xE000_E280 ◀ **ICPR** clear-pending

0xE000_E200 ◀ **ISPR** set-pending

0xE000_E180 ◀ **ICER** clear-enable

0xE000_E100 ◀ **ISER** set-enable, 1 bit per IRQ

0xE000_E100

0xE000_E0FF

SysTick 240 B

24-bit core timer

0xE000_E01C ◀ **CALIB** calibration value

0xE000_E018 ◀ **VAL** current count

0xE000_E014 ◀ **LOAD** reload value

0xE000_E010 ◀ **CTRL** enable, interrupt, clock source

0xE000_E010

0xE000_E00F

Control 16 B

control and ID registers

0xE000_E000

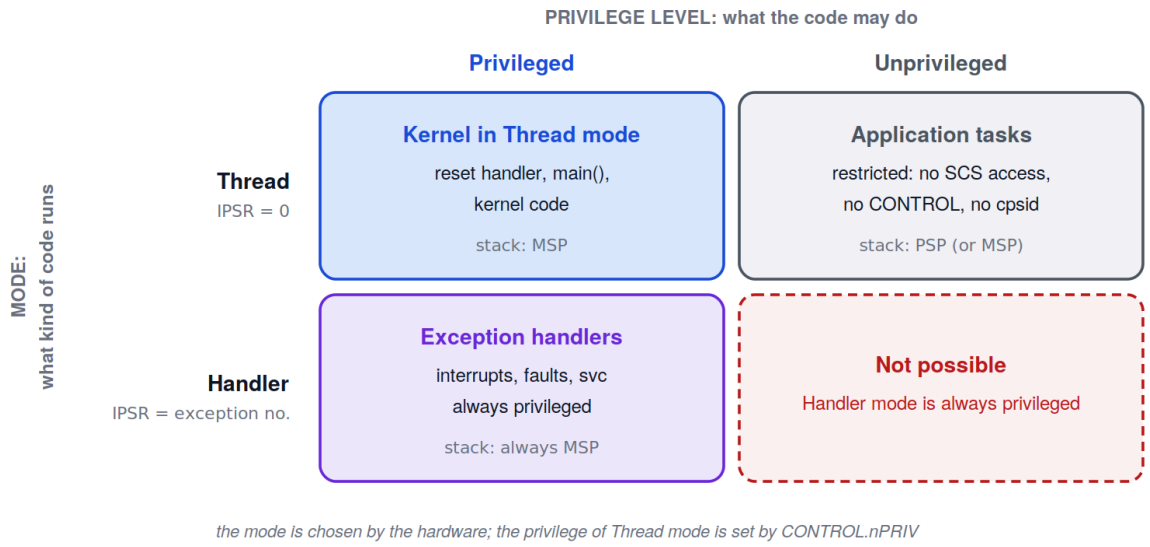
> **IMPORTANT**
>
> Only privileged code may access the SCS. An unprivileged access raises a BusFault, which is what prevents application code from reconfiguring the machine.

4 Execution Modes, Privilege, and Stacks

The Cortex-M3 implements in hardware the **separation** between **kernel mode** and **user mode** found in operating systems. Two independent properties describe the current state:

- the **mode** (what kind of code runs)
- the **privilege level** (what that code may do).

4.1 Thread Mode and Handler Mode



Mode and privilege: *two independent properties of the current state. Three of the four combinations exist; Handler mode is always privileged.*

1. **Thread mode** runs ordinary code: `Reset_Handler`, `main()`, and the functions they call; `IPSR` is 0.
2. **Handler mode** runs exception handlers; `IPSR` holds the exception number. The CPU enters Handler mode automatically when it takes an exception and returns to the previous mode when the handler finishes.

Mode	Privilege	Typical code
Thread	Privileged	`Reset_Handler`, kernel code after reset
Thread	Unprivileged	Application tasks after the kernel drops privilege
Handler	Always privileged	Every exception handler

> **ONLY THREE COMBINATIONS.**
>
> Privilege can be lowered only in Thread mode. Handler mode is always privileged, so an exception always runs privileged and returns to Thread mode with the privilege level it had before.

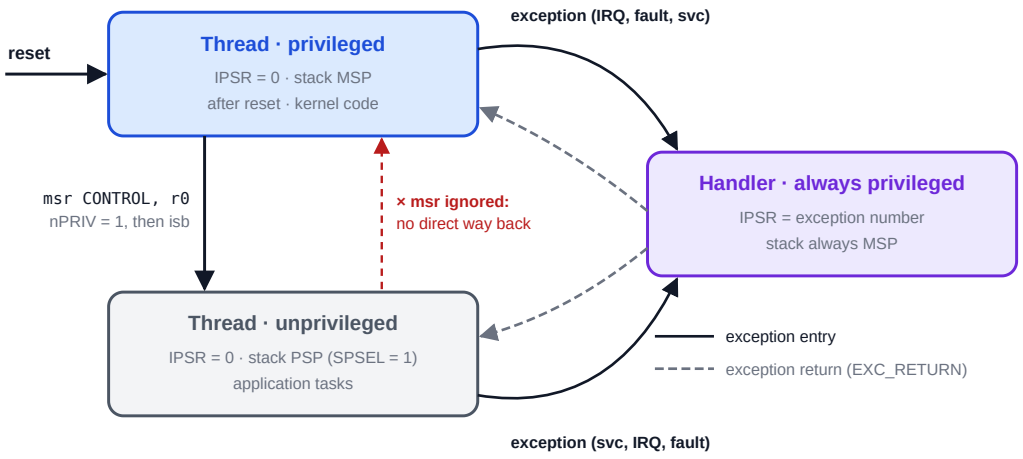


Figure 1.4: *Transitions between privileged Thread mode, unprivileged Thread mode, and Handler mode.*

After reset the CPU is in privileged Thread mode, which is why the boot code can configure the vector table, fault handling, and priorities.

4.2 Dropping Privilege

Privilege is dropped by setting `CONTROL.nPRIV` with the read-modify-write sequence shown in the section on the CONTROL register, followed by `isb`. The change is one-way: an unprivileged `msr control, r0` that tries to clear `nPRIV` is silently ignored, and the only ways back to privileged execution are a reset or an exception, whose handler runs in Handler mode.

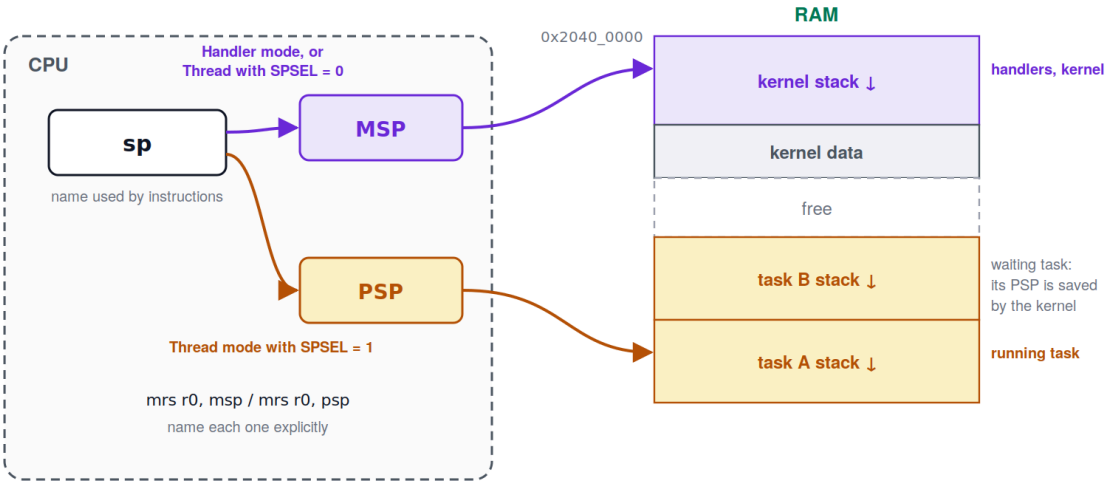
Unprivileged code keeps access to ordinary RAM, Flash, and its own code, but loses the ability to reconfigure or silence the machine:

Attempt by unprivileged code	Result
Access to the SCS (SysTick, NVIC, SCB, MPU)	BusFault
`msr control, r0` to regain privilege	Ignored
`cpsid i` / `cpsie i`	Ignored
Access to a region denied by the MPU	MemManage fault
Semihosting `bkpt 0xab`	Refused by QEMU by default: the fault escalates to HardFault

4.3 Main and Process Stack Pointers

The Cortex-M3 has two physical stack pointers:

1. The **main stack pointer** (`MSP`) is used after reset and always in **Handler mode**; it is the kernel's stack.
2. The **process stack pointer** (`PSP`) is used in **Thread mode** when `CONTROL.SPSEL` is 1; an operating system gives each task its own `PSP` stack. A task therefore typically runs with `CONTROL = 3` : unprivileged and on `PSP` .



an exception always switches to MSP: handlers and system calls never run on a task's stack, so a task that overflows its own stack cannot damage the kernel stack

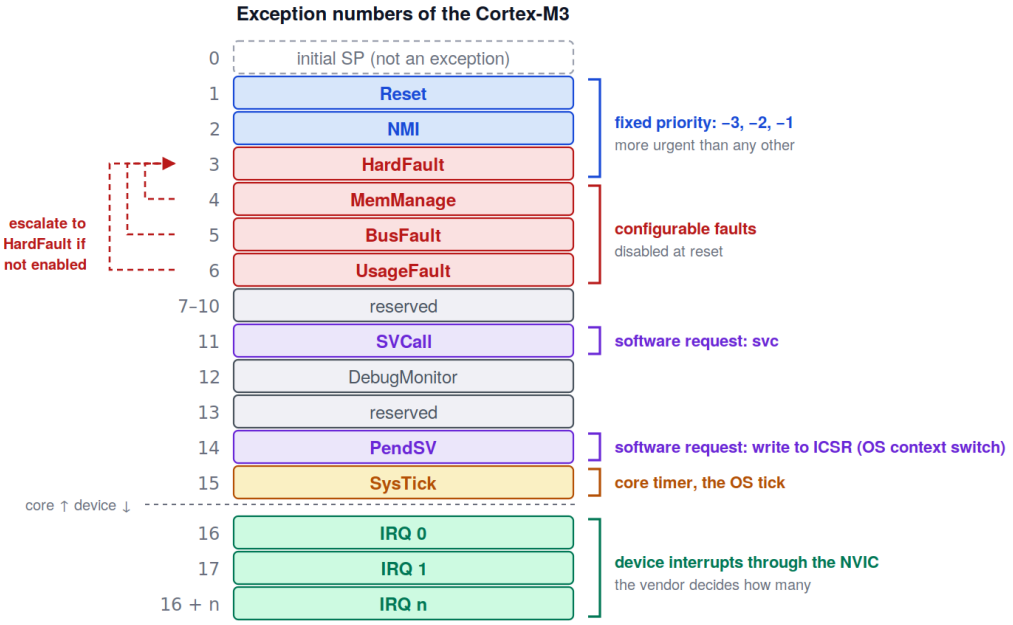
Main and process stack pointers: *sp refers to MSP or PSP depending on the mode and on CONTROL.SPSEL. MSP points to the kernel stack, PSP to the stack of the running task.*

The register name `sp` always refers to the stack pointer currently selected, while `mrs r0, msp` and `mrs r0, psp` name each one explicitly. Because Handler mode always switches to `MSP`, interrupt handlers and system calls never execute on a task's stack: a task that corrupts or overflows its own stack cannot corrupt the stack the kernel runs on.

5 Exception Model

An **exception** is any event that makes the **CPU suspend** the current code and **run a specific handler** in **Handler mode**. An **interrupt** is an **exception** raised by a **hardware device**. Faults, system calls, the system timer, and device interrupts all go through the same mechanism.

5.1 Exception Types and Numbers



Exception numbers: *numbers 1–15 belong to the core and are the same on every Cortex-M3; device interrupts start at 16. Disabled configurable faults escalate to HardFault.*

Number	Exception	Raised by
1	Reset	Power-on, reset pin
2	NMI	Non-maskable interrupt (e.g. watchdog expiry)
3	HardFault	Any fault with nowhere else to go
4	MemManage	Violation of MPU rules
5	BusFault	Access to an address that does not answer or refuses the access
6	UsageFault	Undefined opcode, Thumb bit 0, division by zero (if trapped)
7–10, 13	Reserved	—
11	SVCall	The `SVC` instruction
12	DebugMonitor	Debug events in monitor mode
14	PendSV	A software request (`ICSR.PENDSVSET`)
15	SysTick	The core timer

Number	Exception	Raised by
16 + n	IRQ n	Device interrupt line n, through the NVIC

Numbers 0–15 belong to the core and are identical on every Cortex-M3; device interrupts follow. Number 0 is not an exception: its vector table slot holds the initial stack pointer.

At reset only HardFault is active among the faults: MemManage, BusFault, and UsageFault are disabled, so every fault *escalates* to HardFault until the boot code enables the dedicated handlers.

PendSV deserves a note because of its role in operating systems. It is raised only by software, by writing a bit in a system register, and it is normally given the lowest priority. An OS uses it to request work, typically a context switch, that must run only after every more urgent handler has finished.

5.2 Priorities, Preemption, and Lockup

When several exceptions compete, the more urgent one runs first. On Cortex-M a **lower number means more urgent**. A pending exception preempts the running code only if its priority is *strictly* more urgent; otherwise it stays pending until the current handler returns. Preemption of one handler by another is called *nesting*, which gives the **NVIC (Nested Vectored Interrupt Controller)** its name.

Implementations do not use all eight bits of a priority byte. Real chips implement only the upper 3 or 4 bits, and the lower bits read as 0, so distinct priorities must differ in their top bits (`0x00` , `0x20` , ..., `0xE0` for 3 bits).

> **WARNING**
>
> Lockup If a fault occurs while the CPU already runs at HardFault priority, or the fault cannot be reported because exception entry itself fails (for example, the stack pointer points outside RAM), there is nowhere to escalate. The CPU enters *lockup* and stops executing. QEMU reports it as `Lockup: can't escalate 3 to HardFault (current priority -1)` .

5.3 Vector Table and Reset Sequence

The vector table is an array of 32-bit words where slot N holds the address of the handler of exception N . When exception N occurs, the CPU reads the slot and branches there. Slot 0 is special: it holds the initial value of `MSP` , not a code address. The table is located through the `VTOR` register, which resets to `0x0000_0000` ; at reset it must therefore be at the start of the Code region, which is why the linker places it first in Flash.

When the Cortex-M3 leaves reset, before any software runs, the hardware performs exactly two reads:

1. the word at address `0x0` is loaded into `MSP` ;
2. the word at address `0x4` is loaded into `PC` , and execution starts there in privileged Thread mode.

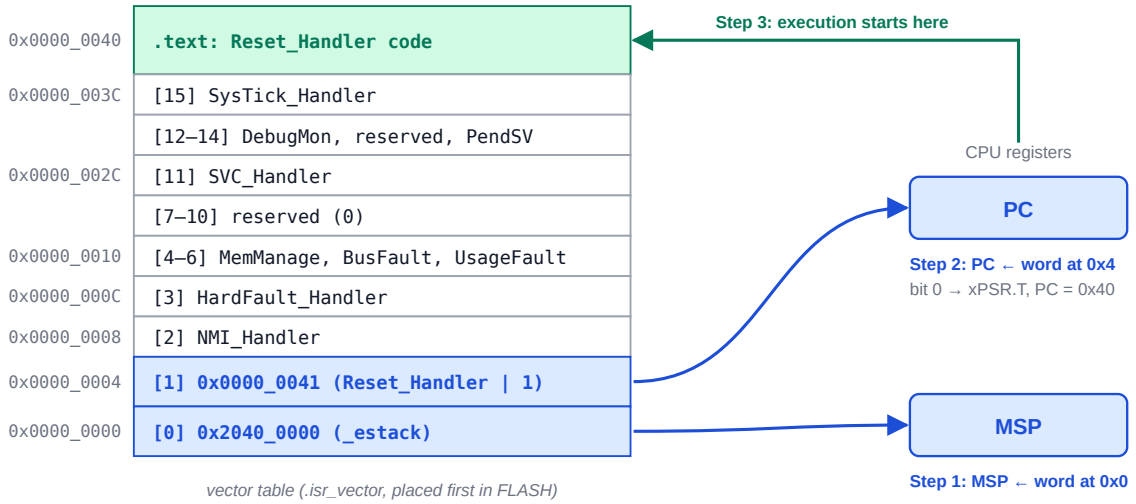


Figure 1.5: *Reset sequence: the CPU loads MSP from address 0x0 and PC from address 0x4, then executes Reset_Handler.*

There is no BIOS, no loader, and no C runtime. Initializing memory, configuring the core, and calling `main()` are the job of the code whose address is stored in slot 1, `Reset_Handler`.

Offset	Slot	Content
`0x00`	0	Initial `MSP` (e.g. `0x2040_0000`, one byte past the end of RAM)
`0x04`	1	`Reset_Handler` address, with bit 0 set
`0x08`	2	`NMI_Handler`
`0x0C` – `0x18`	3–6	HardFault, MemManage, BusFault, UsageFault handlers
`0x2C`	11	`SVC_Handler`
`0x38`, `0x3C`	14, 15	`PendSV_Handler`, `SysTick_Handler`
`0x40` + 4n	16 + n	Handler of IRQ n

Every handler address is stored with the Thumb bit set. Reserved slots hold 0, and every other slot must point to valid code even if the exception is never expected; startup code usually fills unused slots with a default handler that loops forever, so that an unexpected exception stops at a recognizable address.

> **TIP**
>
> From IRQ number to vector slot IRQ n is exception 16 + n, and its handler address is at offset 4 × (16 + n). IRQ 1 is exception 17, at offset `0x44`.

5.4 Exception Entry and the Stacked Frame

Before the first instruction of a handler, the hardware:

1. pushes 8 words, the *stacked frame*, onto the stack in use when the exception occurred (`MSP` or `PSP`);

2. writes a special `EXC_RETURN` value into `lr`;
3. reads the handler address from the vector table and branches to it in Handler mode, on `MSP`.

Offset from new `sp`	Stacked register
`+0x00`	`r0`
`+0x04`	`r1`
`+0x08`	`r2`
`+0x0C`	`r3`
`+0x10`	`r12`
`+0x14`	`lr` of the interrupted code
`+0x18`	`PC`: where the interrupted code resumes
`+0x1C`	`xPSR`

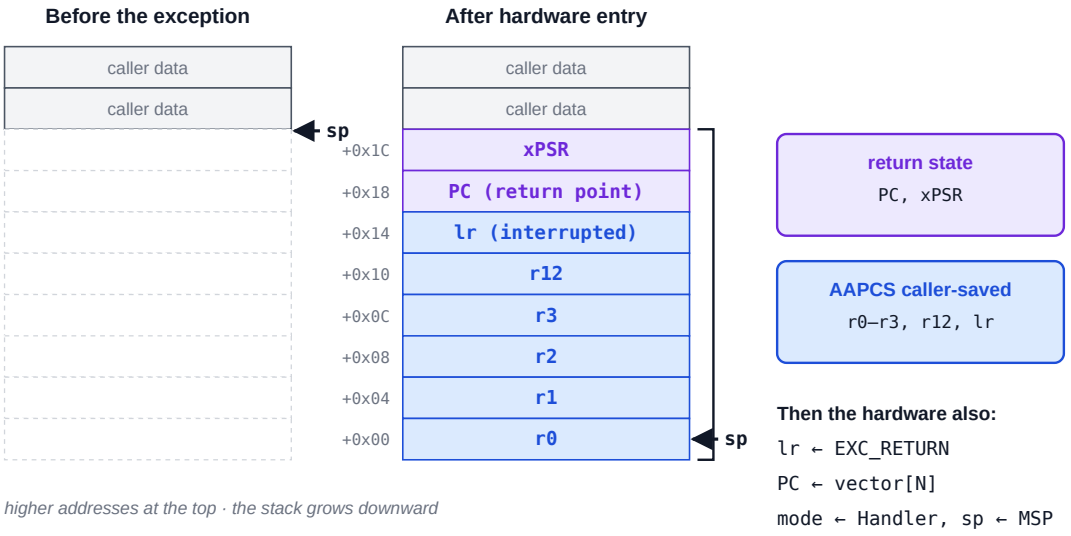


Figure 1.6: *Stack before and after exception entry, with the 8-word stacked frame pushed by hardware.*

The hardware saves exactly the AAPCS caller-saved registers plus `PC` and `xPSR`, and any compiled C function already preserves the callee-saved `r4`–`r11`. Together, these two facts mean that an exception handler can be an ordinary C function, with no special prologue or epilogue. When `CCR.STKALIGN` is set (its reset value on the Cortex-M3), the hardware also aligns `sp` to 8 bytes on entry, as the AAPCS requires, and records the adjustment in bit 9 of the stacked `xPSR`. Entry takes about 12 cycles.

5.5 Exception Return through EXC_RETURN

A handler ends with an ordinary return instruction (`bx lr` or `pop {..., pc}`), yet the CPU must restore the saved state rather than jump to a return address. While a handler runs, `lr` holds a value of the form `0xFFFFFFxx`, where no code can exist. Loading such a value into `PC` is recognized as an *exception return*: the

CPU pops the 8-word frame from the indicated stack, restores the mode, and resumes at the stacked `PC`.

`EXC_RETURN`	Return to	Frame popped from
`0xFFFFFFF1`	Handler mode (a preempted handler)	`MSP`
`0xFFFFFFF9`	Thread mode	`MSP`
`0xFFFFFFFD`	Thread mode	`PSP`

Bits 31–4 are all ones and mark the value as `EXC_RETURN`. Bit 3 selects the return mode (1 = Thread, 0 = Handler) and bit 2 the stack (1 = `PSP`, 0 = `MSP`). On entry, bit 2 therefore tells the handler which stack holds the frame just pushed, which a system-call handler needs in order to read the caller's arguments.

5.6 Tail-Chaining and Late Arrival

When exceptions occur close together, the core avoids saving and restoring the same state repeatedly.

Optimization	Situation	Behaviour
Tail-chaining	A handler ends while another exception is pending	No pop and re-push of the frame: direct jump to the next handler (about 6 cycles instead of about 24)
Late arrival	A more urgent exception arrives while the frame of a less urgent one is being pushed	The frame is reused; the more urgent handler runs first, then tail-chains to the other

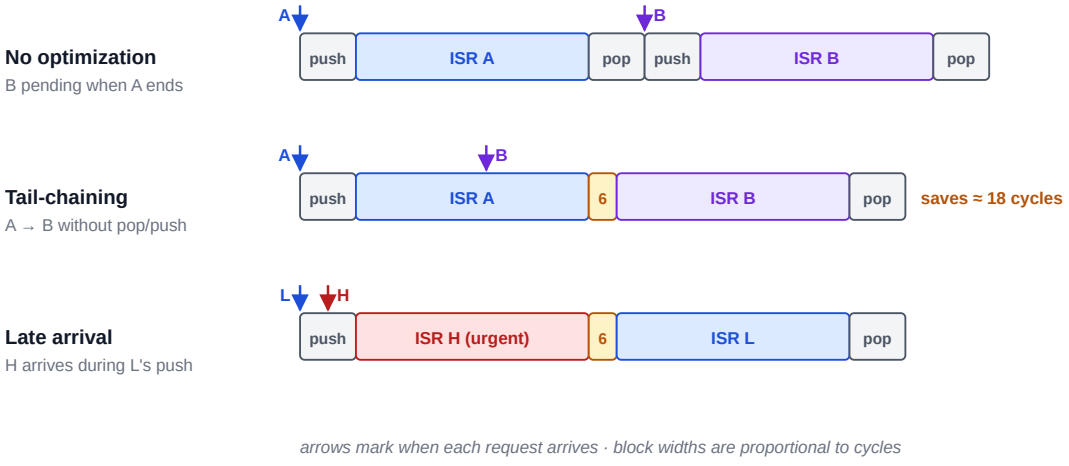


Figure 1.7: *Timelines of two exceptions without optimization, with tail-chaining, and with late arrival.*

5.7 SVC: A Controlled Entry into Privileged Code

```
void SVC_Handler( void )
{
    /* privileged service code */
}
```

```
/* in unprivileged Thread mode */
__asm__ volatile ( "svc #0" : : : "memory" );
```

`svc #n` (supervisor call) is one of the few things unprivileged code can always do. It raises SVCall (exception 11) synchronously, so the handler in slot 11 runs in Handler mode and therefore privileged. When the handler returns, execution resumes at the instruction after `svc`, in Thread mode and still unprivileged. Inside the handler `CONTROL.nPRIV` still reads 1: the handler is privileged because of the mode, not because the bit changed.

Privilege is borrowed only for the duration of the handler, and only code placed in the handler runs with it: the caller can request a service but cannot choose what executes. The immediate `#n` is encoded in the instruction and, together with arguments in `r0`–`r3`, can select the requested service. This is the hardware foundation of every system call: restricted code cannot touch the machine directly and must go through a single, controlled entry point.

> **MISSING HANDLER**
>
> If `SVC_Handler` is not defined, `svc` reaches the default handler and the program hangs silently. Unlike a fault handler, `SVC_Handler` returns normally, because `svc` is a deliberate request, not an error.

6 Configuring the Core: SCB, NVIC, and SysTick

The behaviour of the exception model is configured through memory-mapped registers in the System Control Space. All of them are accessible only from privileged code.

6.1 System Control Block Registers

Register	Address	Role
`CPUID`	`0xE000_ED00`	Core identification (`0x410FC231` : ARM, Cortex-M3)
`ICSR`	`0xE000_ED04`	Pending state of system exceptions; `PENDSVSET` (bit 28) raises PendSV
`VTOR`	`0xE000_ED08`	Address of the vector table
`CCR`	`0xE000_ED14`	Which conditions are treated as faults
`SHPR1` – `SHPR3`	`0xE000_ED18` – `0xE000_ED20`	Priorities of system exceptions
`SHCSR`	`0xE000_ED24`	Enable and status of system handlers
`CFSR`	`0xE000_ED28`	Cause of MemManage, BusFault, UsageFault
`HFSR`	`0xE000_ED2C`	Cause of HardFault
`MMFAR`	`0xE000_ED34`	Address that caused a MemManage fault
`BFAR`	`0xE000_ED38`	Address that caused a BusFault

`VTOR`. On an exception the CPU reads the handler address from `VTOR + 4 × N`. Bits 31–7 hold the table address, which must be aligned to at least 128 bytes (more when many IRQs exist). Writing `VTOR` is redundant when the table is already at 0, but it makes the boot code correct wherever the image is placed: a bootloader at address 0 can start an application stored higher in Flash, which then points `VTOR` at its own table, or a program can copy its table into RAM to replace handlers at run time.

`SHCSR`. Bits 16 (`MEMFAULTENA`), 17 (`BUSFAULTENA`), and 18 (`USGFAULTENA`) enable the dedicated fault handlers. At reset they are 0, so every fault escalates to HardFault and a single handler cannot tell what went wrong. The other bits report whether handlers are active or pending, so the register is updated with read-modify-write (`|= 0x00070000`), never overwritten.

`CCR`. The configuration bits relevant here are listed below.

Bit	Name	Effect when set
3	`UNALIGN_TRP`	Unaligned word and halfword accesses raise a UsageFault
4	`DIV_0_TRP`	Integer division by zero raises a UsageFault
9	`STKALIGN`	8-byte stack alignment on exception entry (1 at reset)

Without `DIV_0_TRP` , `sdiv` and `udiv` silently return 0 when the divisor is 0 and the program continues with a wrong value; setting the bit turns this silent bug into a fault. Unaligned accesses are legal on the Cortex-M3, only

slower, and trapping them can break correct compiler-generated code, so `UNALIGN_TRP` is usually left off. Since `STKALIGN` is already 1, enabling the division trap requires read-modify-write, giving `0x210`.

`SHPR1` – `SHPR3`. Each system exception has one priority byte, so the three registers can be viewed as a 12-byte array starting at `0xE000_ED18`, indexed by exception number minus 4. Each byte is written with a byte store and needs no read-modify-write.

Register	Byte 0	Byte 1	Byte 2	Byte 3
`SHPR1`	MemManage	BusFault	UsageFault	—
`SHPR2`	—	—	—	SVCall
`SHPR3`	—	—	PendSV	SysTick

A typical policy makes faults most urgent (`0x00`), system calls intermediate (`0x80`), and PendSV and SysTick least urgent (`0xE0`). Reading the words as little-endian, this gives `SHPR2 = 0x80000000` and `SHPR3 = 0xE0E00000`.

6.2 Fault Status Registers

```
void UsageFault_Handler( void )
{
    kprintf( "UsageFault: CFSR=0x%x\n", SCB_CFSR );
    for ( ;; ) { }
}

int main( void )
{
    volatile uint32_t numerator = 42;
    volatile uint32_t zero = 0;
    volatile uint32_t result = numerator / zero;
    (void)result;
    for ( ;; ) { }
}
```

`kprintf` is the kernel's formatted print function, built on the UART driver described in the chapter on devices, interrupts, and system calls; before that driver exists, the same experiment can be followed in gdb by stopping in `UsageFault_Handler` and reading `CFSR`.

The divisor is `volatile`, so the compiler cannot evaluate the division at build time and must emit a real `udiv` instruction. With `CCR.DIV_0_TRP` and `SHCSR.USGFAULTENA` set, the instruction raises a UsageFault, the dedicated handler runs, and it prints `CFSR=0x2000000`: bit 25, `DIVBYZERO`. The handler then loops forever, because a fault is a bug with no sensible way to continue. Removing each configuration step isolates its role:

Configuration	Observed behaviour
`SHCSR` bits and `DIV_0_TRP` set	`UsageFault_Handler` runs and reports the cause

Configuration	Observed behaviour
`SHCSR` bits not set	The fault escalates to HardFault; its kind is lost
`DIV_0_TRP` not set	No fault: the division returns 0 and `main` continues

`CFSR` is the concatenation of three status registers, one per configurable fault. Its most useful bits are:

Bits	Sub-register	Main flags
7–0	MMFSR (MemManage)	`IACCVIOL` (0), `DACCVIOL` (1), `MUNSTKERR` (3), `MSTKERR` (4), `MMARVALID` (7)
15–8	BFSR (BusFault)	`PRECISERR` (9), `IMPRECISERR` (10), `BFARVALID` (15)
31–16	UFSR (UsageFault)	`UNDEFINSTR` (16), `INVSTATE` (17), `UNALIGNED` (24), `DIVBYZERO` (25)

`HFSR` bit 30 (`FORCED`) indicates that the HardFault is an escalated configurable fault; the original cause is then found in `CFSR` . The bits are sticky and are cleared by writing 1 to them.

6.3 NVIC: Device Interrupts

```
NVIC->IPR[UART0TX_IRQn] = 0xC0;                              /* priority of IRQ 1 */
NVIC->ISER[UART0TX_IRQn / 32] = 1UL << ( UART0TX_IRQn % 32 ); /* enable IRQ 1 */
```

Device interrupt requests do not reach the CPU directly: they pass through the NVIC, which numbers them, enables them, and assigns their priority. The first line writes the priority byte of IRQ 1 with a byte store. The second sets bit `n % 32` of word `n / 32` in the enable register; since set and clear registers ignore 0 bits, a plain assignment enables one line without disturbing the others.

Register	Base address	Bit or byte for IRQ n	Write semantics
`ISER`	`0xE000_E100`	Bit `n % 32` of word `n / 32`	1 = enable
`ICER`	`0xE000_E180`	Same	1 = disable
`ISPR`	`0xE000_E200`	Same	1 = set pending
`ICPR`	`0xE000_E280`	Same	1 = clear pending
`IPR`	`0xE000_E400`	Byte n	Priority; lower = more urgent

A device interrupt runs its handler only if all of the following hold:

1. the device is configured to raise it (an interrupt-enable bit in the device's own control register);
2. the NVIC line is enabled in `ISER` ;
3. its priority is more urgent than that of the running code, and `PRIMASK` is 0.

NVIC lines driven by device flags are level-sensitive: as long as the device keeps its interrupt flag set, the request stays active. A handler must therefore *acknowledge* the interrupt at the device, typically by writing 1 to clear the flag, otherwise the handler is re-entered forever as soon as it returns.

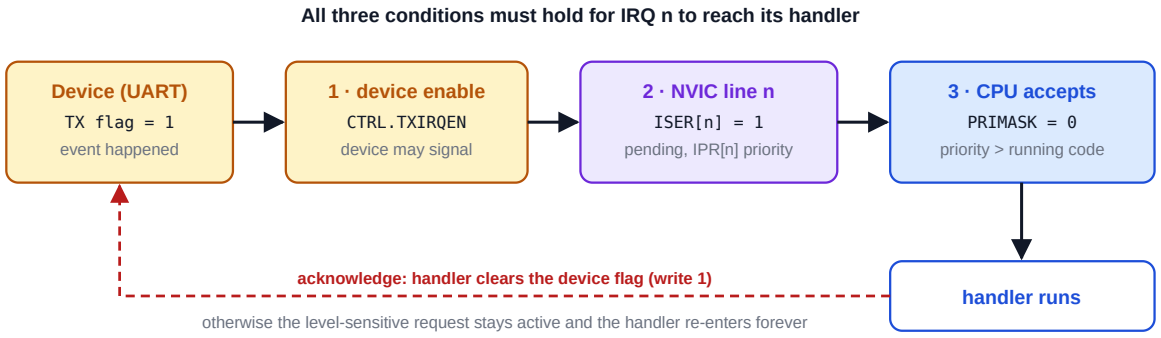


Figure 1.8: *Path of a device interrupt through the device enable, the NVIC, and the CPU masking and priority check, with acknowledgement at the device.*

6.4 SysTick: The Core Timer

SysTick is a 24-bit down-counter present in every Cortex-M core. It counts from the reload value to 0, reloads, and can raise exception 15 each time it reaches 0. An operating system uses this periodic exception, the *tick*, as its time base.

Register	Address	Content
`CTRL`	`0xE000_E010`	`ENABLE` (0), `TICKINT` (1), `CLKSOURCE` (2), `COUNTFLAG` (16, cleared on read)
`LOAD`	`0xE000_E014`	24-bit reload value (max `0xFFFFFF`)
`VAL`	`0xE000_E018`	Current count; any write clears it
`CALIB`	`0xE000_E01C`	Read-only; `TENMS` = reload value for 10 ms on the reference clock

`CLKSOURCE` selects between the processor clock (1) and an implementation-defined reference clock (0). On `mps2-an385` the reference clock runs at 1 MHz (`CALIB.TENMS = 9999`) and the processor clock at 25 MHz. `CALIB` serves portable code that does not know the processor frequency; when the frequency is known, the reload value is computed directly. Writing `CTRL = 7` starts the counter on the processor clock with the exception enabled.

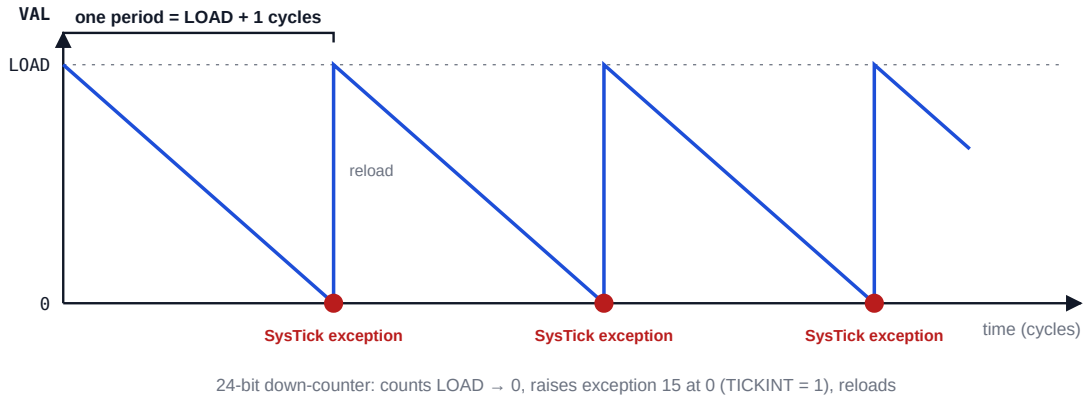


Figure 1.9: *SysTick counter value over time: a sawtooth from LOAD to 0 with an exception at every zero.*

Since one period lasts `LOAD + 1` cycles, the reload value for a tick rate r at clock frequency f is

$$\mathrm{LOAD} = \frac{f}{r} - 1$$

At 25 MHz and 100 ticks per second, `LOAD = 249 999` (`0x3D08F`), which fits in 24 bits; the longest possible period is 2^{24} cycles, about 0.67 s. A faster tick gives finer time resolution at the cost of more exceptions per second.

SysTick is a core exception, not a device interrupt, and this changes how it is configured and handled:

Aspect	SysTick	Device interrupt
Priority set in	`SHPR3`	NVIC `IPR`
Enabled by	`CTRL.TICKINT`	Device enable bit and NVIC `ISER`
Acknowledge in the handler	Not needed: pending state clears on entry	Required, otherwise the handler is re-entered

7 Memory Protection Unit

The Cortex-M3 has no MMU. What it offers instead is an optional **Memory Protection Unit** (MPU), present on `mps2-an385` and disabled after reset.

7.1 MPU Versus MMU

An MMU *translates* every virtual address into a physical one through page tables, and checks permissions as part of the translation. An MPU *only checks*: on every load, store, and instruction fetch, it compares the address with a short list of regions. If the access is allowed, it proceeds at the same physical address; if not, it does not happen at all (a store writes nothing, a load reads nothing) and the CPU raises a MemManage fault. Every task sees real physical addresses, so the linker must place each task at its own address.

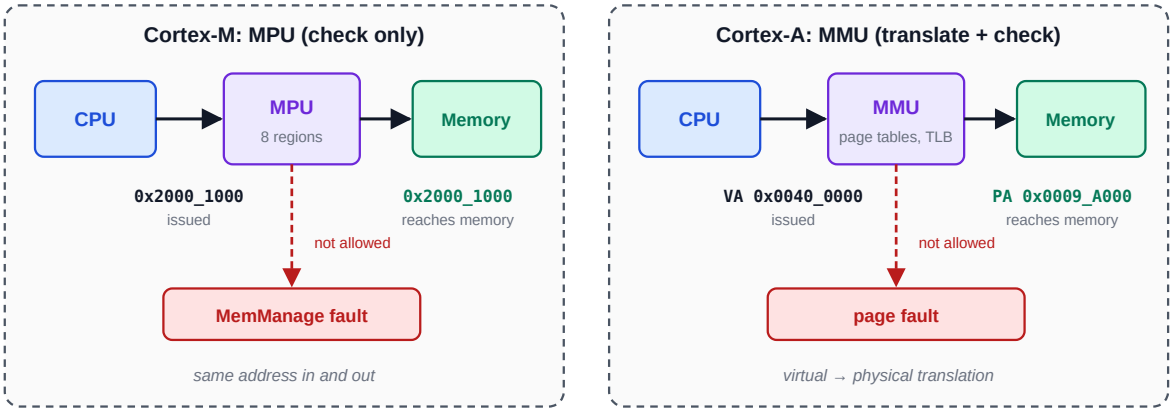


Figure 1.10: *Comparison of an MPU, which only checks addresses, and an MMU, which translates virtual to physical addresses.*

Microcontrollers adopt the MPU for reasons that follow directly from embedded constraints. An MPU is a handful of region registers, while an MMU needs page tables in RAM, a hardware table walker, and a TLB, which cost silicon area and energy. An MPU check takes the same time on every access, whereas a TLB miss makes some accesses slower by an amount that depends on the TLB contents, which harms timing predictability. Finally, with no disk and a small fixed set of tasks, virtual memory would bring no benefit.

7.2 Regions and Permissions

The Cortex-M3 MPU has 8 regions, numbered 0 to 7 (the count is read from `MPU_TYPE` bits 15–8). Each region has a base address, a size, permissions, and an enable bit, subject to two rules:

- the size is a power of two, from 32 bytes to 4 GB;
- the base address is a multiple of the size, i.e. the region is aligned to its own size.

A 4 KB region can start at `0x2000_1000` but not at `0x2000_0400`. A task that needs 5 KB receives an 8 KB region, the next power of two.

The **AP** (access permission) field encodes two rights at once, one for privileged and one for unprivileged code:

AP	Privileged	Unprivileged
`000`	No access	No access
`001`	Read/write	No access
`010`	Read/write	Read-only
`011`	Read/write	Read/write
`110`	Read-only	Read-only

The **XN** (execute never) bit forbids instruction fetches from a region while still allowing reads and writes if AP permits them. Marking data and stack regions XN prevents code injection: a task cannot execute machine code it has written into a buffer.

The **PRIVDEFENA** bit of `MPU_CTRL` decides what happens to privileged accesses that hit no region. When it is 1, they use the default memory map as if the MPU were off, so the kernel keeps access to all memory and peripherals and only unprivileged code is restricted. When it is 0, even privileged code may access only what the regions allow. An unprivileged access that hits no enabled region is always refused.

7.3 MPU Registers and Region Encoding

Register	Address	Fields
`MPU_TYPE`	`0xE000_ED90`	Bits 15–8: number of regions
`MPU_CTRL`	`0xE000_ED94`	`ENABLE` (0), `PRIVDEFENA` (2)
`MPU_RNR`	`0xE000_ED98`	Region selected by `RBAR` and `RASR`
`MPU_RBAR`	`0xE000_ED9C`	Base address (31–5), `VALID` (4), `REGION` (3–0)
`MPU_RASR`	`0xE000_EDA0`	`XN` (28), `AP` (26–24), `S` / `C` / `B` (18–16), `SIZE` (5–1), `ENABLE` (0)

A single `RBAR` / `RASR` pair serves all eight regions; `RNR` selects which one it refers to. When `RBAR` is written with its `VALID` bit set, the region number is taken from its own `REGION` field, so one write replaces the pair of writes to `RNR` and `RBAR`. The low 5 bits of a base address are always 0, because the base is aligned to at least 32 bytes, which leaves room for `VALID` and `REGION` in the same word.

The `SIZE` field holds $\log_2(\text{size}) - 1$, so a region spans $2^{\text{SIZE}+1}$ bytes; the offset of 1 lets 5 bits reach 4 GB with `SIZE = 31`. The `S`, `C`, and `B` bits describe the memory type for caches and multi-core systems and have no functional effect on a core without cache.

```
#define MPU_CTRL (*(volatile uint32_t *)0xE000ED94)
#define MPU_RBAR (*(volatile uint32_t *)0xE000ED9C)
#define MPU_RASR (*(volatile uint32_t *)0xE000EDA0)

void mpu_setup( void )
{
    MPU_RBAR = 0x00000000u | 0x10u | 0u;  /* region 0: Flash, VALID */
    MPU_RASR = 0x0602002Bu;               /* AP=110, C, 4 MB, enabled */
```

```
    MPU_RBAR = 0x20001000u | 0x10u | 1u;  /* region 1: task partition, VALID */
    MPU_RASR = 0x13060017u;               /* XN, AP=011, S, C, 4 KB, enabled */
    MPU_CTRL = ( 1u << 2 ) | 1u;          /* PRIVDEFENA | ENABLE */
    __asm volatile ( "dsb\n\tisb" : : : "memory" );
}
```

The function configures a typical two-region layout for running one unprivileged task. Region 0 covers the 4 MB of Flash as read-only and executable for everyone, so the task can run its code and read constants. Region 1 covers the 4 KB partition of the running task as read/write and non-executable. Everything else (other tasks' memory, kernel data, kernel stack, peripherals) has no region and is therefore forbidden to unprivileged code, while `PRIVDEFENA` leaves the kernel unrestricted. A kernel that switches tasks only needs to rewrite region 1. The final `dsb` waits for all register writes to complete and `isb` ensures that the following instructions are checked against the new regions.

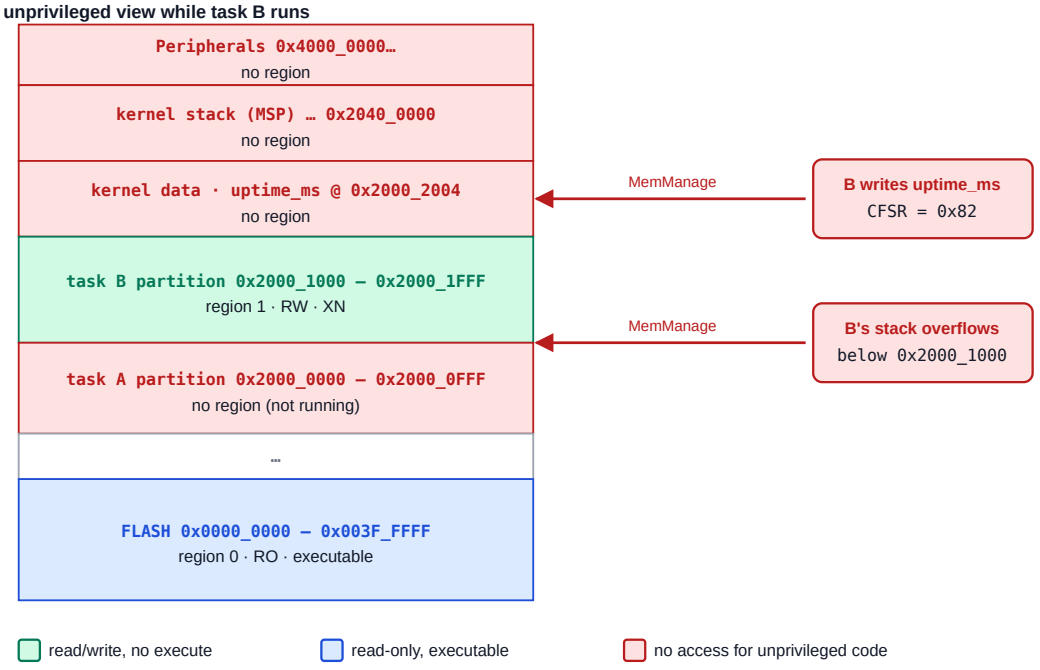
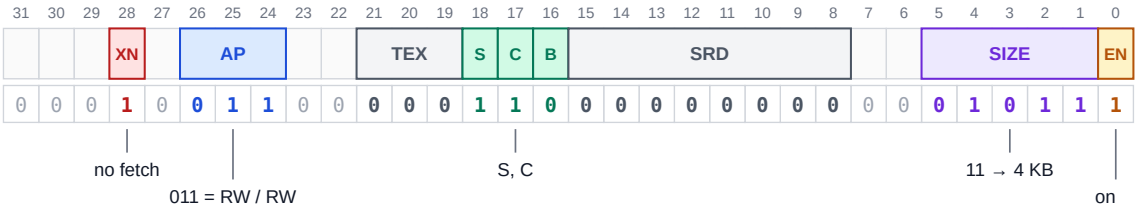


Figure 1.11: *Unprivileged view of memory with the MPU enabled while task B runs: Flash region 0, task B partition region 1, everything else forbidden.*

The two `RASR` values decompose as follows:

Field	Region 0 (Flash)	Region 1 (partition)
`XN` (bit 28)	0	1 → `0x10000000`
`AP` (bits 26–24)	`110` → `0x06000000`	`011` → `0x03000000`
`S`, `C` (bits 18, 17)	`C` → `0x00020000`	`S`, `C` → `0x00060000`
`SIZE` (bits 5–1)	21 (4 MB = 2^{22}) → `0x2A`	11 (4 KB = 2^{12}) → `0x16`
`ENABLE` (bit 0)	`0x01`	`0x01`
Total	`0x0602002B`	`0x13060017`



RASR = 0x1306_0017: region 1 (task partition) · TEX and SRD left at 0

Figure 1.12: *Field layout of MPU_RASR with the value 0x13060017 decoded bit by bit.*

7.4 MemManage Faults

A refused access raises MemManage (exception 4), which reaches its own handler only if `SHCSR.MEMFAULTENA` is set; otherwise it escalates to HardFault. The handler runs privileged and on `MSP`, and it finds the cause in the low byte of `CFSR`:

Bit	Name	Set when
0	`IACCVIOL`	An instruction fetch was refused (e.g. from an XN region)
1	`DACCVIOL`	A data load or store was refused
3	`MUNSTKERR`	Popping the frame on exception return was refused
4	`MSTKERR`	Pushing the frame on exception entry was refused
7	`MMARVALID`	`MMFAR` holds the refused address

Several bits can be set at once. A value of `0x82`, for example, means `MMARVALID` plus `DACCVIOL`: a data access was refused and its address is in `MMFAR`. Looking that address up in the symbol table of the image (with `nm`) names the variable the faulty code tried to touch. A task that overflows its stack below the start of its partition produces the same kind of fault, which makes the MPU an effective stack-overflow detector.

> **WARNING**
>
> Read `MMFAR` only when `MMARVALID` is set For instruction-fetch and stacking faults the CPU does not update `MMFAR`, which may then hold a stale, meaningless value.